Homework 2, CS685 Spring 2025

LINK: https://drive.google.com/file/d/1JxJBuieXhkyXP_92FeqPZaXmCdEiCqrB/view?usp=sharing

Question 1

a) Condition for dead neuron

Given Network:

$$z_i = \mathrm{ReLU}(\theta_i^{(x\to z)} \cdot x + b_i)$$
$$y = \theta^{(z\to y)} \cdot z$$
$$Input = x \in \{0,1\}^D$$

A ReLU unit z_i is dead if the unit's output is zero for every input x, i.e.:

$$\max_{x\in\{0,1\}^D} (\theta_i^{(x\to z)} \cdot x + b_i) \le 0$$

The maximal dot product over binary inputs is achieved by setting $x_d = 1$ for weights > 0 and $x_d = 0$ for weights < 0. Thus a convenient sufficient/necessary condition:

$$b_i + \sum_{d:\theta_{i,d}^{(x\to z)}>0} \theta_{i,d}^{(x\to z)} \le 0$$

So, node z_i is dead iff the above holds (pre-activation never becomes positive).

b) Gradients

For $a_i = \theta_i^{(x\to z)} \cdot x + b_i$ and, $z_i = \mathrm{ReLU}(a_i)$:

$$\frac{\partial \ell}{\partial b_i} = \frac{\partial \ell}{\partial y}\frac{\partial y}{\partial z_i}\frac{\partial z_i}{\partial a_i}\frac{\partial a_i}{\partial b_i}$$

We know $\partial\ell/\partial y = 1$. Also $\partial y/\partial z_i = \theta_i^{(z\to y)}$. Next $\partial z_i/\partial a_i$ equals 1 if $a_i > 0$, and 0 if $a_i < 0$. At $a_i = 0$ subgradient: choose 0 or 1 but standard backprop picks 0 or undefined; we can say subgradient in [0,1]. For typical practical implementations, derivative at 0 is 0.

Finally \partial a_i/\partial b_i = 1.

So:

$$\frac{\partial \ell}{\partial b_i} = \theta^{(z\to y)} i \cdot \mathbf{1} a_i > 0$$

Similarly for weight components \theta^{(x\to z)}_{j,i} (weight on input feature j for hidden unit i):

$$\frac{\partial \ell}{\partial \theta_{j,i}^{(x\to z)}} = \frac{\partial \ell}{\partial y} \cdot \theta^{(z\to y)} i \cdot \mathbf{1} a_i > 0 \cdot x_j$$

Given \partial\ell/\partial y=1, that simplifies to

$$\frac{\partial \ell}{\partial \theta_{j,i}^{(x\to z)}} = \theta^{(z\to y)} i \cdot \mathbf{1} a_i > 0 \cdot x_j$$

If the unit is inactive on that instance $(a_i \le 0)$, the indicator = 0 so both gradients are zero.

c) Why dead neurons never recover

If the neuron is dead, by definition for every training input x we have $a_i = \theta^{(x\to z)} i \cdot x + b_i \le 0$. For any such example the ReLU derivative $\mathbf{1} a_i > 0 = 0$. Therefore from (b) the gradients $\partial\ell/\partial b_i$ and $\partial\ell/\partial\theta_{j,i}^{(x\to z)}$ are zero for every training instance (except measure-zero boundary cases where a_i=0 and subgradient choice matters). If the gradient is zero across all training points, gradient descent will not change b_i or $\theta_i^{(x\to z)}$. So the pre-activation remains nonpositive for all inputs — the neuron stays dead.

Question 2

RNN single hidden unit $h_m = \sigma(\theta h_{m-1} + x_m)$. Show if $|\theta| < 1$ then $\partial h_m/\partial h_{m-k} \to 0$ as $k \to \infty$.

The chain-rule expression:

$$\frac{\partial h_m}{\partial h_{m-k}} = \prod_{t=1}^{k} \sigma'(\theta h_{m-t} + x_{m-t+1}) \cdot \theta.$$

Each term is bounded in magnitude. For sigmoid

$$\sigma(s), \sigma'(s) = \sigma(s)(1 - \sigma(s)) \leq 1/4$$

So,

$$\left| \frac{\partial h_m}{\partial h_{m-k}} \right| \leq \prod_{t=1}^{k} |\theta| \cdot \max_s \sigma'(s) = (|\theta| \cdot C)^k$$

with $C = \max_s \sigma'(s) \leq 1/4$, so $|\theta|C < 1$ whenever $|\theta| < 4$. In particular if $|\theta| < 1$, then $|\theta|C < 1$ and the product tends to zero exponentially in k. Hence gradient vanishes.

Question 3

3.1 Dimensionalities

We have linear softmax layer $q = \text{softmax}(y)$ with $y = Wx$. Vocabulary V.

W is $|V| \times m$ (rows: one score per word; columns: dimension of context embedding $x \in \mathbb{R}^m$).
y is $|V|$-dimensional vector (pre-softmax scores or prefix vector representation).
q is $|V|$-dimensional (probability distribution over vocabulary).

3.2 Gradient for cross-entropy

Cross-entropy loss with one-hot target for word d is:

$L = -\log q_d$

Softmax: $q_k = \frac{e^{y_k}}{\sum_j e^{y_j}}$

Compute derivative elementwise. For component i of y:

$$\frac{\partial L}{\partial y_i} = -\frac{1}{q_d} \cdot \frac{\partial q_d}{\partial y_i}.$$

But known result:

$$\frac{\partial q_d}{\partial y_i} = \begin{cases} q_d(1 - q_d) & i = d \\ \bullet q_d q_i & i \neq d \end{cases}$$

So, for $i = d$:

$$\frac{\partial L}{\partial y_d} = -\frac{1}{q_d} \cdot q_d(1 - q_d) = q_d - 1.$$

For $i \neq d$:

$$\frac{\partial L}{\partial y_i} = -\frac{1}{q_d} \cdot (-q_d q_i) = q_i.$$

Thus the compact vector form:

$\nabla_y L = q - e_d$,

where e_d is the one-hot vector with 1 at index d.

3.3 Interpretation

The gradient $q - e_d$ pushes the model to increase y_d (because gradient for y_d is $q_d - 1$, negative when $q_d < 1$) and decrease other y_i proportional to their current probability q_i. In expectation (over data distribution), the gradient is zero at optimum when the model probabilities match empirical distribution. Taking an SGD step subtracts (learning rate times) this gradient, shifting pre-softmax scores to put more mass on observed data words. This matches the high-level goal: cross-entropy will increase probability on observed word and redistribute probability mass from words that currently have probability mass.

3.4

We need the whole vector y in backprop because each y_i contributes to the softmax denominator. Even though loss depends directly only on q_d, but q_d depends on all y_j. The Jacobian $\partial q_d / \partial y$ has nonzero entries for all j. Therefore backprop requires the full gradient $\nabla_y L$, not just the scalar derivative w.r.t. y_d.

Computation graph sketch:

$$x \to y = Wx \to q = \text{softmax}(y) \to L(q)$$
$$\text{The gradient } \partial L / \partial W = (\partial L / \partial y) x^T \text{— so we need } \partial L / \partial y = q - e_d$$

Question 4

4.1 Implemententing Cosine Similarity with GloVe word Embeddings

```
pip install gensim
```

```
Requirement already satisfied: gensim in /usr/local/lib/python3.12/dist-packages (4.3.3)
Requirement already satisfied: numpy<2.0,>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.26.4)
Requirement already satisfied: scipy<1.14.0,>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.13.1
Requirement already satisfied: smart-open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.3.1)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart-open>=1.8.1->gensim) (1.
```

```
import gensim.downloader as api
import numpy as np

glove = api.load("glove-wiki-gigaword-50")
vocab = list(glove.index_to_key)
W = np.vstack([glove[w] for w in vocab])
W_norm = W / np.linalg.norm(W, axis=1, keepdims=True)

def nearest_neighbors(word, K=10):
    v = glove[word] / np.linalg.norm(glove[word])
    sims = W_norm @ v
    idx = np.argsort(-sims)
    res = []
    for i in idx:
        if vocab[i]==word: continue
        res.append((vocab[i], float(sims[i])))
        if len(res)>=K: break
    return res
```

```
[==================================================] 100.0% 66.0/66.0MB downloaded
```

4.2 Nearest words and Words that make sense

```
print("dog neighbors:")
dog = nearest_neighbors("dog", K=10)
for i in dog: print(i)
print("cat neighbors:")
cat = nearest_neighbors("cat", K=10)
for i in cat: print(i)
```

```
dog neighbors:
('cat', 0.9218005537986755)
('dogs', 0.8513159155845642)
('horse', 0.7907583713531494)
('puppy', 0.7754921913146973)
('pet', 0.7724707722663879)
('rabbit', 0.7720813751220703)
('pig', 0.7490061521530151)
('snake', 0.7399188280105591)
('baby', 0.7395570874214172)
('bite', 0.7387937903404236)
cat neighbors:
('dog', 0.9218005537986755)
('rabbit', 0.8487821221351624)
('monkey', 0.804108202457428)
('rat', 0.7891963124275208)
('cats', 0.7865270376205444)
('snake', 0.7798910737037659)
('dogs', 0.7795814871788025)
('pet', 0.7792249917984009)
('mouse', 0.773166835308075)
('bite', 0.7728800177574158)
```

Some of the neighbors feels like connected to each other. For the word dog, words like cat, horse, and rabbit fit perfectly since they are animals in a different form and are to some extent considered as household animals. The same goes for cat being close to dog, horse, rabbit all animals you'd expect to show up in similar conversations. Also, words like dogs and cats are basically plural forms of respective words so make sense that they are close. Moreover, the word pet make sense coz both are pet animals and puppy coz their babies are refered as puppies. These words make sense because the model groups words that often appear in the same kind of context, like pets or household animals.

4.3 Words that make less sense

A few of the neighbors are a bit odd. For example, dog being close to baby or bite, and cat being linked with bite, monkey, or snake, don't really feel like true "neighbors." This happens because the embeddings are built from patterns in text — so if "dog bite" or "cat bite" show up a lot, those words get pulled together even though they aren't the same kind of thing. In other words, the model is picking up on co-occurrence in language, not just whether two words actually belong to the same category.

4.4 Nearest Neighbor for "hai"

```
print("hair neighbors:")
hair = nearest_neighbors("hair", K=10)
for i in hair: print(i)
```

```
hair neighbors:
('blond', 0.7977585792541504)
('skin', 0.7930654883384705)
('dyed', 0.7885915637016296)
('wears', 0.7828461527824402)
('beard', 0.7704828381538391)
('pink', 0.7636144161224365)
('glasses', 0.762917160987854)
('socks', 0.7550185322761536)
('pants', 0.7520442605018616)
('mask', 0.7380977272987366)
```

I tested the word hair, I got some neighbors that were very directly connected, like blond, dyed, beard, mask and pink. These clearly make sense because they are describing either the color, condition, or a closely related feature to hair itself. For example, blond and pink show how embeddings capture common hair colors that appear in text, while dyed reflects the process of changing hair, and beard is naturally associated with facial hair, and mask is because of hair-mask. These results show that the model has picked up on the way people often talk about hairstyles, colors, and related body features in similar contexts. What I found interesting were the less direct neighbors, such as skin, glasses, socks and pants. At first glance these don't really seem like they belong in the same category as hair. But when you think about it, they actually show how word embeddings are sensitive to descriptive contexts in language. For instance, in everyday text you might see sentences like "she has blond hair and wears glasses" or "he has dark hair, pale skin, and a beard." Similarly, clothing words like socks or pants appear because people often list multiple physical descriptors together — hair color, what someone is wearing, and other visible features. So the embedding doesn't just group words by strict dictionary meaning, it groups them by the company they keep in real sentences. This is a good example of how embeddings go beyond pure semantics and capture broader descriptive associations. The close neighbors tie directly to hair, while the stranger ones reflect the way hair often gets mentioned alongside appearance and clothing in text.

Question 5

5.1 Code Implementation

```
terms = ["doctor","receptionist","engineer","scientist","nurse","teacher"]

def gender_score(word, male="he", female="she"):
    g = glove[male] - glove[female]
    g = g / np.linalg.norm(g)
    v = glove[word] / np.linalg.norm(glove[word])
    return float(np.dot(v,g))

scores = [(t, gender_score(t)) for t in terms]
for word, score in sorted(scores, key=lambda x: -x[1]):
    print(word," : ",score)
```

```
engineer  :  0.12540802359580994
scientist  :  0.03890509530901909
teacher  :  -0.10364638268947601
doctor  :  -0.12392482161521912
receptionist  :  -0.36983543634414673
nurse  :  -0.3813399076461792
```

Here, roles such as engineer and scientist show positive scores, indicating male-associated language, while professions like nurse and receptionist have strongly negative scores, reflecting female-associated language. Teacher and doctor show mild female bias, which tells that the text corpora may encode shifting perceptions compared to historical norms.

5.2 For other terms

```
terms = ["artist","poet","musician","chef","student","professor", "coach", "athlete", "soldier", "dancer", "model",
for t in terms:
    print(t, " : ", gender_score(t))
```

```
artist  :  -0.23729616403579712
poet  :  0.011249118484556675
musician  :  -0.09934014827013016
chef  :  -0.1829930990934372
student  :  0.014502441510558128
professor  :  0.08265193551778793
coach  :  0.3562123775482178
athlete  :  -0.059179261326789856
soldier  :  -0.04333103820681572
dancer  :  -0.3573048710823059
model  :  -0.09547163546085358
designer  :  -0.23055805265903473
writer  :  -0.09220660477876663
```

I selected a set of terms from creative, professional, and athletic roles to explore how word embeddings encode implicit gender associations. The roles like coach (+0.356) lean strongly male, while dancer (−0.357), artist (−0.237), and designer (−0.231) lean strongly female which shows the common societal stereotypes about these professions. Others, such as professor (+0.083), poet (+0.011), and student (+0.015), show near-neutral scores which shows more gender-balanced perceptions in the embedding space. Similarly, musician (−0.099) and chef (−0.183) exhibit mild female bias, while athlete (−0.059) and soldier (−0.043) are only slightly female-associated, indicating weaker or more context-dependent biases. The result highlights how some roles show strong alignment with traditional stereotypes, whereas others are more neutral or counter-stereotypical. This emphasizes the potential for NLP models to carry implicit gender biases, even in contexts where the associations may seem subtle, which could influence downstream applications such as hiring recommendation systems, automated content generation, or text classification.

5.3 Seed Terms

```
seed_pairs = [("he","she"), ("man","woman"), ("male","female")]
def compute_scores_for_seeds(terms, seeds):
    out = {}
    for male,female in seeds:
        g = glove[male] - glove[female]
        g = g / np.linalg.norm(g)
        out[(male,female)] = [(t, float(np.dot(glove[t]/np.linalg.norm(glove[t]), g))) for t in terms]
    return out

res = compute_scores_for_seeds(terms, seed_pairs)
for k,v in res.items():
    print("Seeds:", k)
    for t,s in sorted(v, key=lambda x:-x[1]):
        print(f"  {t}: {s:.4f}")
    print()
```

```
Seeds: ('he', 'she')
  coach: 0.3562
  professor: 0.0827
  student: 0.0145
  poet: 0.0112
  soldier: -0.0433
  athlete: -0.0592
  writer: -0.0922
  model: -0.0955
  musician: -0.0993
  chef: -0.1830
  designer: -0.2306
  artist: -0.2373
  dancer: -0.3573

Seeds: ('man', 'woman')
  coach: 0.3534
  chef: 0.0541
  writer: -0.0010
  professor: -0.0102
  poet: -0.0193
  musician: -0.0235
  designer: -0.0277
  model: -0.0343
  soldier: -0.0630
  athlete: -0.0691
  artist: -0.0968
  student: -0.1477
  dancer: -0.1902

Seeds: ('male', 'female')
  athlete: -0.0232
  model: -0.0780
  dancer: -0.1311
  soldier: -0.1438
  coach: -0.1753
  poet: -0.1784
  professor: -0.1860
  student: -0.2151
```

```
designer: -0.2265
musician: -0.2409
artist: -0.2503
chef: -0.3044
writer: -0.3512
```

If we look at the three sets of seed words, there is a clear difference in how stable or unstable the results are depending on the choice of seeds. When the gender direction is defined with the pairs *he–she* or *man–woman*, the patterns are broadly consistent. In both cases, engineer and scientist come out on the male-associated side, while teacher, receptionist, and nurse are female-associated, with doctor leaning slightly negative as well. The magnitudes of the scores shift a bit for example, doctor is somewhat closer to neutral in the *he–she* case but more female coded with *man–woman*—yet the overall ranking is similar. These results align closely with well-documented gender stereotypes in language data, where technical or scientific professions are associated with men, and service or care professions are linked with women.

This changes drastically when the gender direction is defined with *male–female*. Here the results are flipped, all six professions are now assigned negative values, meaning they are female-associated. Here the rank ordering is overturned. Engineer, which had been the most male ssociated profession under the other two seed choices, now drops to the most female- ssociated position. Doctor, which had previously leaned female, becomes the least female associated, occupying the top of the ranking. This instability shows how sensitive the method is to the exact words chosen to represent gender.

The shift suggests that the embedding space encodes "male/female" differently than "he/she" or "man/woman." While pronouns and common nouns are used in everyday contexts and capture relational associations between genders and social roles, the adjectives *male* and *female* are often used in more formal, biological, or categorical contexts (e.g., "male patient," "female candidate"). Because the embeddings reflect usage patterns in large text corpora, the neighborhood around *male/female* emphasizes these descriptive or categorical contexts, which may not align with social stereotypes about professions in the same way. As a result, professions no longer line up with the stereotypical male/female division seen with the other seed pairs.

So the choice of seed words is not a neutral methodological step but one that can reshape the results quite substantially. When pronouns or basic human nouns are used, the outcomes seem more robust and stereotype-consistent, but with different lexical choices the analysis can break down or even invert. This makes it clear that any conclusions drawn from gender direction analysis must be interpreted with caution, since they are in part an artifact of the seeds chosen to define the concept of gender in the embedding space.

5.4 Gender Bias in Word Embeddings

Yes, gender bias in word embeddings matters, and it matters in ways that are both practical and ethical. Word embeddings are widely used as building blocks in systems such as search engines, recommendation systems, hiring algorithms, chatbots, and even decision support tools in sensitive domains like healthcare or education. If these embeddings encode biased associations for example, linking "engineer" more closely to men and "nurse" more closely to women, then those stereotypes are carried forward into the applications that rely on them. What seems like a small statistical skew in a vector space can become a structural inequity when it systematically shapes how systems rank candidates, recommend content, or frame possibilities for users.

The problem goes beyond the risk of "offensive outputs." Bias in embeddings reflects deep patterns in the data they are trained on news articles, Wikipedia, web text sources that already carry the imprint of historical and societal inequalities. As Blodgett et al. (2020) emphasize, these biases are not just technical artifacts but evidence of how social hierarchies and stereotypes are reproduced through data. If researchers and practitioners treat embeddings as neutral mirrors of language, then they risk naturalizing these inequalities, reinforcing them instead of questioning them. This is particularly dangerous because embeddings often operate invisibly within systems; users may not realize that biased associations are shaping the information they see or the opportunities made available to them.

One might argue that embeddings simply reflect the real world and that stereotypes in the data are unavoidable. But this view ignores the fact that representation in language is itself a form of power. If the embedding space encodes women as less associated with science or leadership, then a model built on those embeddings is not neutrally reflecting reality. It is amplifying a distorted view of who belongs in those roles. Moreover, "reflecting reality" is not the same as "justifying reality", society changes, and part of the role of technology should be to enable more equitable futures rather than to lock in the prejudices of the past.

5.5 Extra Credit

```
import gensim.downloader as api
import numpy as np
from sklearn.cluster import KMeans

glove = api.load("glove-wiki-gigaword-100")
vocab = list(glove.index_to_key)
terms = ["doctor","receptionist","engineer","scientist","nurse","teacher"]

def gender_score(word, male="he", female="she"):
    g = glove[male] - glove[female]
    g = g / np.linalg.norm(g)
    v = glove[word] / np.linalg.norm(glove[word])
```

```
    return float(np.dot(v,g))

def neutralize(word, gender_direction):
    v = glove[word]
    gender_component = (np.dot(v, gender_direction) / np.linalg.norm(gender_direction)**2) * gender_direction
    return v - gender_component

male, female = "he", "she"
g = glove[male] - glove[female]
g = g / np.linalg.norm(g)
debiased_vecs = {t: neutralize(t, g) for t in terms}
print("=== Gender scores before debiasing ===")
for t in terms:
    print(t, ":", gender_score(t))

print("\n=== Gender scores after debiasing (should be ~0) ===")
for t in terms:
    v = debiased_vecs[t] / np.linalg.norm(debiased_vecs[t])
    print(t, ":", float(np.dot(v, g)))

X = np.array([debiased_vecs[t] for t in terms])
kmeans = KMeans(n_clusters=2, random_state=42).fit(X)
labels = kmeans.labels_
print("\n=== K-means clustering on debiased embeddings ===")
for t, label in zip(terms, labels):
    print(f"{t:12s} -> Cluster {label}")
```

```
[==================================================] 100.0% 128.1/128.1MB downloaded
=== Gender scores before debiasing ===
doctor : -0.06612934917211533
receptionist : -0.27680736780166626
engineer : 0.14717267453670502
scientist : 0.06814918667078018
nurse : -0.2951718270778656
teacher : -0.09193938970565796

=== Gender scores after debiasing (should be ~0) ===
doctor : 1.5832483768463135e-08
receptionist : -3.4924596548080444e-10
engineer : 2.9103830456733704e-09
scientist : -4.307366907596588e-09
nurse : -9.992049854190554e-09
teacher : 4.540197551250458e-09

=== K-means clustering on debiased embeddings ===
doctor       -> Cluster 1
receptionist -> Cluster 1
engineer     -> Cluster 0
scientist    -> Cluster 0
nurse        -> Cluster 1
teacher      -> Cluster 1
```

Before debiasing, the professions show clear gender associations: engineer and scientist align with male, while nurse, receptionist, teacher, and even doctor lean female. After applying the neutralization step, all cosine similarities with the gender direction collapse to nearly zero. On the surface, this suggests the embeddings have become gender-neutral and that bias has been effectively removed. However, when I applied Gonen and Goldberg's k-means test, a different picture emerges. Despite their "neutral" scores, the professions still cluster into two groups that closely follow gender stereotypes. Engineer and scientist remain grouped together, while nurse, receptionist, teacher, and doctor fall into a separate cluster. This indicates that the deeper geometric relationships between words still encode gender, even though the direct gender axis was eliminated.

These findings support the criticism that projection-based debiasing creates only a cosmetic solution. It hides bias along one chosen dimension but does not alter the embedding space in a way that truly removes gendered structure. As a result, gender remains detectable through indirect methods like clustering, meaning that biased patterns can still propagate into downstream tasks. In short, your results confirm Gonen and Goldberg's claim that debiasing does not equal de-biasing — the information is still there, just less obvious.

Question 6

6.1 BOW Implementation

```
import re
import numpy as np
from datasets import load_dataset
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
```

```
dd = load_dataset("imdb")
train = dd['train']
test = dd['test']

def tokenize(text):
    return [t for t in re.split(r'\W+', text.lower()) if t]

X_train_texts = [" ".join(tokenize(t)) for t in train['text']]
y_train = np.array(train['label'])
X_test_texts = [" ".join(tokenize(t)) for t in test['text']]
y_test = np.array(test['label'])
vectorizer = CountVectorizer(lowercase=False, token_pattern=r"(?u)\b\w+\b")
X_train_bow = vectorizer.fit_transform(X_train_texts)
X_test_bow = vectorizer.transform(X_test_texts)
# logistic regression (L2)
clf_bow = LogisticRegression(max_iter=1000, solver='liblinear')
clf_bow.fit(X_train_bow, y_train)
y_pred_bow = clf_bow.predict(X_test_bow)
print("BOW test accuracy:", accuracy_score(y_test, y_pred_bow))
print(classification_report(y_test, y_pred_bow, digits=4))
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/toke
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
README.md: 7.81k/? [00:00<00:00, 930kB/s]
plain_text/train-00000-of-00001.parquet: 100% 21.0M/21.0M [00:00<00:00, 28.4MB/s]
plain_text/test-00000-of-00001.parquet: 100% 20.5M/20.5M [00:00<00:00, 110kB/s]
plain_text/unsupervised-00000-of-00001.p(…): 100% 42.0M/42.0M [00:00<00:00, 30.8MB/s]
Generating train split: 100% 25000/25000 [00:00<00:00, 92752.36 examples/s]
Generating test split: 100% 25000/25000 [00:00<00:00, 189415.46 examples/s]
Generating unsupervised split: 100% 50000/50000 [00:00<00:00, 203439.42 examples/s]
BOW test accuracy: 0.86932
              precision    recall  f1-score   support

           0     0.8639    0.8767    0.8703     12500
           1     0.8749    0.8619    0.8683     12500

    accuracy                         0.8693     25000
   macro avg     0.8694    0.8693    0.8693     25000
weighted avg     0.8694    0.8693    0.8693     25000
```

6.2 Results of BOW

The bag-of-words logistic regression classifier achieved an overall **test accuracy of 86.93%**. Precision and recall were balanced across both positive and negative sentiment classes. Specifically, for the negative class, precision was **0.8639** and recall was **0.8766**, while for the positive class, precision was **0.8748** and recall was **0.8619**. The F1-scores were **0.8702** and **0.8683** for the negative and positive classes respectively, giving an overall weighted F1-score of **0.8693**. These results show that the classifier is performing consistently well across both classes, without a strong bias toward predicting one sentiment over the other. Compared to the manual keyword-based heuristic baseline from class, the logistic regression BOW model clearly performs better, capturing richer lexical information from the training data and generalizing more effectively to unseen reviews.

6.3 BOE Implementation

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
import gensim.downloader as api
glove = api.load("glove-wiki-gigaword-100")
embed_dim = glove.vector_size

def tokenize(text):
    return [t for t in re.split(r'\W+', text.lower()) if t]

def doc_average_embedding(text):
    toks = tokenize(text)
    vecs = [glove[t] for t in toks if t in glove]
    if len(vecs) == 0:
        return np.zeros(embed_dim, dtype=float)
    return np.mean(vecs, axis=0)
```

```
X_train_boe = np.vstack([doc_average_embedding(t) for t in train['text']])
X_test_boe = np.vstack([doc_average_embedding(t) for t in test['text']])
clf_boe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000, solver='liblinear'))
clf_boe.fit(X_train_boe, y_train)
y_pred_boe = clf_boe.predict(X_test_boe)
print("BOE test accuracy:", accuracy_score(y_test, y_pred_boe))
print(classification_report(y_test, y_pred_boe, digits=4))
```

```
BOE test accuracy: 0.80136
              precision    recall  f1-score   support

           0     0.7975    0.8078    0.8026     12500
           1     0.8053    0.7949    0.8001     12500

    accuracy                         0.8014     25000
   macro avg     0.8014    0.8014    0.8014     25000
weighted avg     0.8014    0.8014    0.8014     25000
```

6.4 Results of BOE

On the IMDB test set, the bag-of-embeddings (BOE) classifier with GloVe embeddings reached an accuracy of 80.14%, whereas the bag-of-words (BOW) logistic regression model achieved a significantly higher accuracy of **86.93%**. This shows that in this large scale setting, the BOW model outperforms the embedding-based approach. The BOW classifier benefits from learning directly from the raw training data, capturing sentiment bearing words like “amazing,” “awful,” “boring,” or “excellent” without smoothing them together. With 25,000 training examples, logistic regression on the sparse BOW features can assign strong, class-specific weights to these words. On the other hand, the BOE model relies on pretrained GloVe embeddings, where each document is represented by the average of its word vectors. While embeddings encode general semantic similarity, this averaging process tends to dilute sentiment polarity. For example, averaging “great” with neutral words like “movie” and “story” can weaken the strength of sentiment representation. As a result, the BOE classifier struggles to match the precision and recall of BOW, even though it leverages pretrained semantic knowledge. A large dataset like IMDB, BOW logistic regression is more effective than BOE with GloVe embeddings, because the BOW model can directly exploit high frequency sentiment words.

6.5 Explaination

When the training dataset is very small, a bag-of-embeddings (BOE) classifier has an inherent advantage over a bag-of-words (BOW) classifier because it leverages pretrained semantic knowledge. In a BOW model, each word is treated as an independent feature, with no relationship to other words. If the dataset is small, many words that are important for sentiment classification may appear only a few times, or not at all, in the training set. This sparsity means the model cannot reliably learn weights for rare words, even if they carry strong sentiment. For example, if “terrific” only appears twice in the small training sample, the BOW model cannot confidently associate it with positive sentiment. While, a BOE model uses pretrained embeddings such as GloVe, which already encode semantic similarity between words. Even if the word “terrific” is rare in the training set, its vector will be close to words like “great” or “excellent,” which the classifier may have seen more often. Averaging embeddings across a document thus provides a smoother, semantically informed representation that captures sentiment cues beyond just raw frequency. This ability to generalize across related words helps BOE perform better than BOW when labeled training data is scarce.

6.6 Testing given Hypothesis

```
import random
random.seed(42)

indices = list(range(len(train['text'])))
sample_idx = random.sample(indices, 100)
X_small_texts = [train['text'][i] for i in sample_idx]
y_small = y_train[sample_idx]
X_small_joined = [" ".join(tokenize(t)) for t in X_small_texts]
vectorizer_small = CountVectorizer(lowercase=False, token_pattern=r"(?u)\b\w+\b")
X_small_bow = vectorizer_small.fit_transform(X_small_joined)
X_test_bow_small = vectorizer_small.transform([" ".join(tokenize(t)) for t in test['text']])
clf_bow_small = LogisticRegression(max_iter=1000, solver='liblinear')
clf_bow_small.fit(X_small_bow, y_small)
y_pred_bow_small = clf_bow_small.predict(X_test_bow_small)
acc_bow_small = accuracy_score(y_test, y_pred_bow_small)
X_small_boe = np.vstack([doc_average_embedding(t) for t in X_small_texts])
clf_boe_small = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000, solver='liblinear'))
clf_boe_small.fit(X_small_boe, y_small)
y_pred_boe_small = clf_boe_small.predict(X_test_boe)
acc_boe_small = accuracy_score(y_test, y_pred_boe_small)

print("BOW (100) test acc:", acc_bow_small)
```

```
print("BOE (100) test acc:", acc_boe_small)
```

```
BOW (100) test acc: 0.63248
BOE (100) test acc: 0.70492
```

When I cut the training set down to just 100 reviews, the performance of the two models changed quite a bit compared to when we trained on the full dataset. The bag-of-words (BOW) model only reached about **63% accuracy** on the test set, which is only a little better than random guessing. This makes sense, because with such a tiny sample the BOW vocabulary is really limited, and a lot of important sentiment words in the test set never even show up in training. That leaves the model without much useful information to work with. The bag-of-embeddings (BOE) model, on the other hand, did noticeably better, hitting around **70% accuracy**. The reason is that it uses pretrained GloVe embeddings, which already capture relationships between words. Even if a word didn't appear in those 100 training reviews, its embedding is still close to other words with similar meaning, so the classifier can make smarter guesses. For example, if "fantastic" never appeared in training, the model can still pick up that it's positive because its vector is near "great" or "excellent." So with really small training sets, BOE has a real advantage over BOW because it brings in outside knowledge that helps fill the gaps. But when we go back to using the full dataset, BOW actually comes out ahead, since with enough data it can learn very strong, direct associations between words and sentiment.

6.7 Extra Credit

```
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
import numpy as np
import re
from numpy.linalg import norm

pos_words = ["great", "excellent", "fantastic", "amazing", "wonderful"]
neg_words = ["bad", "awful", "boring", "terrible", "worst"]

def tokenize(text):
    return [t for t in re.split(r'\W+', text.lower()) if t]

def avg_embedding(words, emb, dim=100):
    vecs = [emb[w] for w in words if w in emb]
    return np.mean(vecs, axis=0) if vecs else np.zeros(dim)

pos_vec = avg_embedding(pos_words, glove)
neg_vec = avg_embedding(neg_words, glove)

def doc_features(text):
    toks = tokenize(text)
    doc_vec = avg_embedding(toks, glove)
    pos_sim = float(np.dot(doc_vec, pos_vec) / (norm(doc_vec)*norm(pos_vec)+1e-9))
    neg_sim = float(np.dot(doc_vec, neg_vec) / (norm(doc_vec)*norm(neg_vec)+1e-9))
    return np.concatenate([doc_vec, [pos_sim, neg_sim]])

X_train = np.vstack([doc_features(t) for t in dd['train']['text']])
y_train = np.array(dd['train']['label'])
X_test = np.vstack([doc_features(t) for t in dd['test']['text']])
y_test = np.array(dd['test']['label'])
clf = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000, solver="liblinear"))
clf.fit(X_train, y_train)
acc = clf.score(X_test, y_test)
print("BOE + keyword similarity test accuracy:", acc)
```

```
BOE + keyword similarity test accuracy: 0.80268
```

I tried adding the keyword-based features to the bag-of-embeddings model, the test accuracy went up a tiny bit compared to BOE alone. My plain BOE model had about **80.1%**, while **BOE + keyword similarity** reached about **80.3%**. So the improvement is pretty small, but it's there. This actually makes sense. The embeddings already capture a lot of general semantic information, so adding keyword based sentiment signals doesn't completely change the game, but it does give the classifier an extra nudge in the right direction. For example, if a review doesn't have many strong sentiment words, the embedding average might look "neutral," but the keyword similarity features can help highlight subtle cues by comparing the review to our hand-picked positive and negative word lists. Here, combining human knowledge with machine-learned representations can provide modest but consistent benefits. Even if the jump isn't dramatic, it shows how manual features and pretrained embeddings can complement each other, making the model a little bit more sensitive to sentiment cues.

6.8 Extra Credit

```
from datasets import load_dataset
from transformers import DistilBertTokenizerFast, DistilBertForSequenceClassification
from transformers import Trainer, TrainingArguments
import numpy as np
from sklearn.metrics import accuracy_score

dd = load_dataset("imdb")
tokenizer = DistilBertTokenizerFast.from_pretrained("distilbert-base-uncased")
def tokenize(batch):
    return tokenizer(batch['text'], truncation=True, padding="max_length", max_length=128)

train_enc = dd['train'].map(tokenize, batched=True, batch_size=16)
test_enc = dd['test'].map(tokenize, batched=True, batch_size=16)

train_enc.set_format("torch", columns=["input_ids", "attention_mask", "label"])
test_enc.set_format("torch", columns=["input_ids", "attention_mask", "label"])
model = DistilBertForSequenceClassification.from_pretrained("distilbert-base-uncased", num_labels=2)

training_args = TrainingArguments(
    output_dir="./results",
    eval_strategy="epoch",
    save_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=2,
    weight_decay=0.01,
    logging_dir='./logs',
    logging_steps=100
)

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    return {"accuracy": accuracy_score(labels, preds)}

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_enc,
    eval_dataset=test_enc,
    compute_metrics=compute_metrics,
)

trainer.train()
results = trainer.evaluate()
print(results)
```

```
Map: 100%                                   25000/25000 [00:11<00:00, 2003.76 examples/s]

Map: 100%                                   25000/25000 [00:11<00:00, 2453.40 examples/s]

Some weights of DistilBertForSequenceClassification were not initialized from the model checkpoint at distilbert-bas
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
[6250/6250 14:14, Epoch 2/2]
```

Epoch	Training Loss	Validation Loss	Accuracy
1	0.311500	0.346309	0.869400
2	0.234400	0.421700	0.875840

```
[3125/3125 01:36]
{'eval_loss': 0.42170047760009766, 'eval_accuracy': 0.87584, 'eval_runtime': 96.8348, 'eval_samples_per_second': 258
```

I tried improving the model using DistilBERT, a pretrained transformer that produces contextualized embeddings for each review. Unlike bag-of-words or bag-of-embeddings models, DistilBERT considers the full context of words, which helps it capture more nuanced sentiment cues like negations (“not good”) or multi-word expressions. I fine-tuned DistilBERT on the IMDB training set for two epochs. The training went smoothly, and the evaluation on the test set showed an **accuracy of about 87.6%**. This is a clear improvement over the plain BOE model (80.1%) and even slightly better than the BOW logistic regression (86.9%) we used earlier. The results demonstrate the power of contextual embeddings, they not only encode word meaning but also the way words interact in context, making the model more sensitive to subtle cues in the text. While the improvement over BOW isn’t huge in this case, it’s consistent and shows that modern transformer models can outperform traditional approaches even with the same training data. Combining pretrained models with fine-tuning gives the best performance here, capturing both semantic knowledge and context that simpler models miss.

Start coding or generate with AI.